



Team Reference Document

Far Eastern FU: alekseevichi

Максим Сказкин, Никита Кержаков, Денис Старцев

Contents	
1 Contest	1
1.1 Template	1
2 Structures	2
2.1 Disjoint Set Union	2
2.2 PBDS Tree	2
2.3 Segment Tree	2
2.4 Lazy Segment Tree	2
2.5 Fenwick Tree	3
2.6 2D Fenwick Tree	3
2.7 RMQ	4
3 Sorting	4
3.1 Mergesort	4
3.2 Radixsort	4
3.3 Quickselect	4
3.4 Binary search	4
4 Strings	4
4.1 Prefix function	4
4.2 Z function	4
4.3 Manacker algorithm	4
4.4 Hashing	4
4.5 Prefix Tree	4
5 Graphs	4
5.1 Dijkstra Algorithm	4
5.2 Shortest Path Faster	4
5.3 Floyd-Warshall	4
5.4 Euler Path	4
5.5 Hamilton Path	4
5.6 Topological Sort	4
5.7 Stronly Connected Components	4
5.8 Bridge Edges & Cut Vertices	4
6 Math	4
6.1 Equations	4
6.2 LU Decomposition	4
6.3 Polynomials	4
6.4 Extremums	4
6.5 Sums	4
6.6 Trigonometry	4
6.7 Number theory	4
6.8 Combinatorics	11
7 Geometry	11
7.1 Points	11
7.2 Lines	12
7.3 Circles	12
7.4 Polygons	13
7.5 Hull	13
7.6 Some Formulas	14
7.7 Line Containers	14
8 Dynamic Programming	15
8.1 Longest Increasing Subsequence	15
9 Various	15
9.1 Some numbers	15
9.2 Bit Manipulations	15
9.3 Cycle Detection	16
1 Contest	
1.1 Template	
<pre>4 #pragma GCC optimize("O2,unroll-loops") 4 #pragma GCC target("avx2,lzcnt,popcnt") 5 5 #include <bits/stdc++.h> 5 using namespace std; 5 5 using ll = long long; 5 using ull = unsigned long long; 6 using pii = pair<int, int>; 6 using pll = pair<ll, ll>; 6 using vi = vector<int>; 6 using vs = vector<string>; 6 using vb = vector<bool>; 7 using vvi = vector<vector<int>>; 7 using vpai = vector<pair<int, int>>; 7 #define hset unordered_set 7 #define hmap unordered_map 7 #define PB push_back 8 #define EB emplace_back 8 #define MP make_pair 8 #define MT make_tuple 8 #define F first 8 #define S second 8 #define SP ' ' 8 #define NL '\n' 9 #define all(c) (c).begin(), (c).end() 9 9 ios::sync_with_stdio(false); 9 cin.tie(0); cout.tie(0); 9 cin.exceptions(cin.failbit); 9 9 Link streams with files 11 freopen("input.txt", "r", stdin); 11 freopen("output.txt", "w", stdout);</pre>	

Optimization

- * target("03") – auto-vectorizing, inlining, unrolling loops
- * target("0fast") – 03 + some **unsafe** math (beware of it)
- * optimize("trapv") – debug int overflows

Use `__attribute__((target("..."), optimize("...")))` to enable for the function only

Doubles

```
a == b or abs(a - b) < max(abs(a), abs(b)) * numeric_limits<double>::epsilon()
a == b or abs(a - b) < abs(min(a, b)) * numeric_limits<double>::epsilon() * error_factor
output << fixed << setprecision(int) << ...
```

2 Structures

2.1 Disjoint Set Union

Disjoint sets. If undo isn't needed, skip `st`, `time()`, `undo()`. **TC:** $O(1)$ **SC:** $O(n)$

```
struct DSU {
    vi e;
    vector<pii> st;
    DSU(int n) : e(n, -1) {}
    int size(int x) { return -e[find(x)]; }
    int time() { return st.size(); }
    int find(int x) { return e[x] < 0 ? x : find(e[x]); }
    bool join(int a, int b) {
        a = find(a), b = find(b);
        if (a == b) return false;
        if (e[a] > e[b]) swap(a, b);
        st.push_back({a, e[a]});
        st.push_back({b, e[b]});
        e[a] += e[b];
        e[b] = a;
        return true;
    }
    void undo(int t) {
        for (int i = time(); i --> t;)
            e[st[i].first] = st[i].second;
        st.resize(t);
    }
};
```

2.2 PBDS Tree

Set/map with two additional ops. **TC:** $O(\log n)$ **SC:** $O(n)$

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
```

```
// Value = null type for set
typedef __gnu_pbds::tree<Key, Value, less<Key>, __gnu_pbds::rb_tree_tag,
    __gnu_pbds::tree_order_statistics_node_update> OrdTree;
```

- * iterator `find_by_order(size_t ord)`
- * `size_t order_of_key(T elem)`
- * `void push_back(T elem)`

- * `T pop_back()`
- * `void insert(int pos, rope targ)`
- * `void erase(int pos, int count)`
- * `rope substr(int pos, int count)`
- * `void replace(int pos, int count, rope targ)`
- * `rope operator+(rope lhs, rope rhs)`

2.3 Segment Tree

Zero-indexed. Segments exclude end: $[a, b)$. `tree[0]` isn't used. **TC:** $O(\log n)$ **SC:** $O(n)$

```
struct Segtree {
    typedef int T;
    static const T unit = INT32_MIN;
    vector<T> tree;
    int size;
    static T f(T a, T b) {
        return max(a, b);
    };
    Segtree(int sz, T def = unit) : size(__bit_ceil(sz)), tree(2 * sz, def) {}
    Segtree(vector<T> &data) {
        size = __bit_ceil(data.size());
        tree.resize(2 * size);
        copy(data.begin(), data.end(), tree.begin() + size);
        for (int i = size - 1; i > 0; --i) tree[i] = f(tree[2 * i], tree[2 * i + 1]);
    }
    T get(int i) {
        return tree[size + i];
    }
    T query(int lb, int rb) {
        T lf = unit, rf = unit;
        for (lb += size, rb += size; lb < rb; lb /= 2, rb /= 2) {
            if (lb % 2) lf = f(lf, tree[lb++]);
            if (rb % 2) rf = f(tree[--rb], rf);
        }
        return f(lf, rf);
    }
    void update(int i, T val) {
        tree[i + size] = f(tree[i + size], val);
        for (i += size; i /= 2;) tree[i] = f(tree[2 * i], tree[2 * i + 1]);
    }
    void set(int i, T val) {
        for (tree[i += size] = val; i /= 2;) tree[i] = f(tree[2 * i], tree[2 * i + 1]);
    }
};
```

2.4 Lazy Segment Tree

The same, but also can update a whole segment. In following code: `query()` returns minimum, `update()` increments values of a segment **TC:** $O(\log n)$ **SC:** $O(n)$

```
struct LazySegtree {
    typedef int T;
    vector<T> tree, lazy;
    int size;
    LazySegtree(vector<T> &data) : size(__bit_ceil(data.size())) {
```

```

tree.resize(2 * size, INT32_MAX);
lazy.resize(2 * size);
copy(data.begin(), data.end(), tree.begin() + size);
for (int i = size - 1; i > 0; --i) tree[i] = min(tree[2 * i], tree[2 * i + 1]);
}
T query(int ql, int qr, int node = 1, int lb = 0, int rb = -1) {
    if (rb == -1) rb = size;
    if (qr <= lb or rb <= ql) return INT32_MAX;
    if (ql <= lb && rb <= qr) return tree[node];
    push(node, lb, rb);
    int md = lb + (rb - lb + 1) / 2;
    return min(query(ql, qr, 2 * node, lb, md), query(ql, qr, 2 * node + 1, md, rb));
}
void update(int ql, int qr, T x, int node = 1, int lb = 0, int rb = -1) {
    if (rb == -1) rb = size;
    if (qr <= lb or rb <= ql) return;
    if (ql <= lb && rb <= qr) {
        lazy[node] += x;
        tree[node] += x;
    } else {
        push(node, lb, rb);
        int md = lb + (rb - lb + 1) / 2;
        update(ql, qr, x, 2 * node, lb, md);
        update(ql, qr, x, 2 * node + 1, md, rb);
        tree[node] = min(tree[2 * node], tree[2 * node + 1]);
    }
}
void push(int node, int lb, int rb) {
    if (lazy[node]) {
        int md = lb + (rb - lb + 1) / 2;
        update(lb, rb, lazy[node], 2 * node, lb, md);
        update(lb, rb, lazy[node], 2 * node + 1, md, rb);
        lazy[node] = 0;
    }
}
};

```

2.5 Fenwick Tree

Zero-indexed. Segments include end: $[a, b]$. $tree[0]$ isn't used. Counting data is one-indexed: $tree[i]$ keeps $f(data[i - (i \& -i) + 1, i])$, $data[i]$ is counted in all $i := i + (i \& -i)$.

TC: $O(\log n)$ **SC:** $O(n)$

```

struct Fentree {
    typedef int T;
    vector<T> tree;
    int size;
    Fentree(int sz, T def = 0) : size(sz), tree(sz + 1) {
        for (int i = 1; i <= size; ++i) tree[i] = def * (i & -i);
    }
    Fentree(vector<T> &data) : size(data.size()), tree{0, data[0]} {
        tree.reserve(size + 1);
        vector<T> pref;
        pref.reserve(size); pref.push_back(data[0]);
        for (int i = 2; i <= data.size(); ++i) {
            pref.push_back(pref.back() + data[i - 1]);
            tree.push_back(pref.back());
        }
    }
};

```

```

        if (i != __bit_floor(i)) tree.back() -= pref[i - (i & -i) - 1];
    }
}
T get(int i) { return query(i, i); }
T query(int rb) {
    int res = 0;
    for (rb += 1; rb > 0; rb -= (rb & -rb)) res += tree[rb];
    return res;
}
T query(int lb, int rb) { return query(rb) - query(lb - 1); }
void update(int i, T val) {
    for (i += 1; i <= size; i += (i & -i)) tree[i] += val;
}
void set(int i, T val) { update(i, val - get(i)); }
void append(T val) {
    tree.push_back(val);
    int rb = tree.size() - 1;
    int lb = rb - (rb & -rb) + 1;
    if (1 <= lb and lb <= rb - 1) tree.back() += query(lb, rb - 1);
}
int upper_bound(T x) {
    int i = 0;
    for (int pw = __bit_floor(tree.size()); pw >= 1; pw >>= 1) {
        if (i + pw < tree.size() and tree[i + pw] <= x) {
            i += pw;
            x -= tree[i];
        }
    }
    return i;
}
};

```

2.6 2D Fenwick Tree

The same, but on the plane. **TC:** $O(\log n \log m)$ **SC:** $O(nm)$

```

struct Fenwick2D {
    typedef int T;
    vector<vector<T>> tree;
    int y_size, x_size;
    Fenwick2D(int y_sz, int x_sz)
        : y_size(y_sz), x_size(x_sz), tree(y_sz + 1, vector<T>(x_sz + 1, 0)) {}
    T get(int y, int x) { return query(y, y, x, x); }
    T query(int yr, int xr) {
        if (yr < 0 or xr < 0) return 0;
        T res = 0;
        for (int iy = yr + 1; iy > 0; iy -= iy & -iy)
            for (int ix = xr + 1; ix > 0; ix -= ix & -ix) res += tree[iy][ix];
        return res;
    }
    T query(int yl, int yr, int xl, int xr) {
        return query(yr, xr) - query(yl - 1, xr) - query(yr, xl - 1) + query(yl - 1, xl - 1);
    }
    void update(int y, int x, T val) {
        for (int iy = y + 1; iy <= y_size; iy += iy & -iy)
            for (int ix = x + 1; ix <= x_size; ix += ix & -ix) tree[iy][ix] += val;
    }
    void set(int y, int x, T val) { update(y, x, val - get(y, x)); }
};

```

2.7 RMQ

Query minimum on any range in $O(1)$. Preproc: **TC:** $O(n \log n)$ **SC:** $O(n \log n)$

```
struct RMQ {
    typedef int T;
    vector<vector<T>> seg;
    RMQ(vector<T> &data) : seg(1, data) {
        for (int pw = 1, k = 1; 2 * pw <= data.size(); pw *= 2, ++k) {
            seg.emplace_back(data.size() - 2 * pw + 1);
            for (int i = 0; i < seg.back().size(); ++i)
                seg[k][i] = min(seg[k - 1][i], seg[k - 1][i + pw]);
        }
    }
    T query(int lb, int rb) {
        int pw = 31 - __builtin_clz(rb - lb);
        return min(seg[pw][lb], seg[pw][rb - (1 << pw)]);
    }
};
```

3 Sorting

3.1 Mergesort

Merge sort. **TC:** $O(n \log n)$ **SC:** $O(n)$

```
void mergesort(vi &nums, vi &buff, int lb, int rb) {
    if (rb - lb <= 1) return;
    int md = lb + (rb - lb) / 2;
    mergesort(nums, buff, lb, md);
    mergesort(nums, buff, md, rb);
    copy(nums.begin() + lb, nums.begin() + md, buff.begin() + lb);
    auto lhs = buff.begin() + lb, lhs_end = buff.begin() + md;
    auto rhs = nums.begin() + md, rhs_end = nums.begin() + rb;
    auto target = nums.begin() + lb;
    while (lhs != lhs_end or rhs != rhs_end) {
        if (lhs == lhs_end) *target = *(rhs++);
        else if (rhs == rhs_end) *target = *(lhs++);
        else if (*lhs <= *rhs) *target = *(lhs++);
        else *target = *(rhs++);
        target++;
    }
}
```

3.2 Radixsort

Decimal radix sort. **TC:** $O(n)$ **SC:** $O(n)$

```
void radixsort(vi &nums) {
    if (nums.size() <= 1) return;
    int max_rank = 1;
    for (int num : nums) max_rank = max(max_rank, (int)ceil(log10(abs(num) + 1)));
    vi sorted(nums.size());
    for (int rank = 0, power = 1; rank < max_rank; ++rank, power *= 10) {
        vi digit_count(10, 0);
        for (int num : nums) {
            int digit = (num / power) % 10; digit_count[digit] += 1;
        }
        for (int i = 0, digit_count_accum = 0; i < 10; ++i) {
```

```
            int tmp = digit_count[i];
            digit_count[i] = digit_count_accum;
            digit_count_accum += tmp;
        }
        for (int num : nums) {
            int digit = (num / power) % 10;
            sorted[digit_count[digit]] = num;
            digit_count[digit]++;
        }
        nums = sorted;
    }
}
```

3.3 Quickselect

Select k order statistic. **TC:** $O(n)$ **SC:** $O(1)$

```
int quickselect(vi &nums, int k, int lb, int rb) {
    if (rb - lb == 1) return nums[lb];
    int pivot_idx = lb + rng() % (rb - lb);
    int pivot = nums[pivot_idx];
    swap(nums[lb], nums[pivot_idx]);
    int eq = lb, gr = lb + 1;
    for (int i = lb + 1; i < rb; ++i) {
        int ii = i;
        if (nums[ii] <= pivot) {
            swap(nums[gr], nums[ii]); ii = gr++;
        }
        if (nums[ii] < pivot) swap(nums[eq++], nums[ii]);
    }
    if (k < eq - lb) return quickselect(nums, k, lb, eq);
    if (k < gr - lb) return pivot;
    return quickselect(nums, k - (gr - lb), gr, rb);
}
```

3.4 Binary search

First index to insert element to keep array sorted. **TC:** $O(\log n)$ **SC:** $O(1)$

```
int lower_bound(vi &nums, int target) {
    int lb = 0, rb = nums.size();
    while (lb < rb) {
        int md = lb + (rb - lb) / 2;
        if (nums[md] < target) lb = md + 1; // <= for upper_bound
        else rb = md;
    }
    return lb;
}
```

4 Strings

4.1 Prefix function

Given string s , $\forall i$ finds max n s.t. $s[:n] == s[i-n+1 : i+1]$. **TC:** $O(n)$ **SC:** $O(n)$

```
vi pref_func(string &s) {
    vi pref(s.size(), 0);
    for (int i = 1; i < s.size(); ++i) {
        int c = pref[i - 1];
```

```

    while (c > 0 and s[i] != s[c]) c = pref[c - 1];
    if (s[i] == s[c]) pref[i] = c + 1;
}
return pref;
}
vi pref_func(string &s, string &t) {
    vi t_pref(t.size(), 0), pref(s.size(), 0);
    for (int i = 1; i < t.size(); ++i) {
        int c = t_pref[i - 1];
        while (c > 0 and t[i] != t[c]) c = t_pref[c - 1];
        if (t[i] == t[c]) t_pref[i] = c + 1;
    }
    if (!s.empty() and !t.empty()) pref[0] = (s[0] == t[0]) ? 1 : 0;
    for (int i = 1; i < s.size(); ++i) {
        int c = pref[i - 1];
        while (c > 0 and s[i] != t[c] or c == t.size()) c = t_pref[c - 1];
        if (s[i] == t[c]) pref[i] = c + 1;
    }
    return pref;
}

```

4.2 Z function

Given string s , $\forall i$ finds max n s.t. $s[:n] == s[i : i + n]$. **TC:** $O(n)$ **SC:** $O(n)$

```

vi z_func(string &s) {
    int lb = 0, rb = 0;
    vi z(s.size(), 0);
    for (int i = 1; i < s.size(); ++i) {
        if (i <= rb) z[i] = min(z[i - lb], rb - i + 1);
        while (i + z[i] < s.size() and s[i + z[i]] == s[z[i]]) z[i]++;
        if (i + z[i] - 1 > rb) lb = i, rb = i + z[i] - 1;
    }
    return z;
}
vi z_func(string &s, string &t) {
    int lb = 0, rb = 0;
    vi tz(t.size(), 0), z(s.size(), 0);
    for (int i = 1; i < t.size(); ++i) {
        if (i <= rb) tz[i] = min(tz[i - lb], rb - i + 1);
        while (i + tz[i] < t.size() and t[i + tz[i]] == t[tz[i]]) tz[i]++;
        if (i + tz[i] - 1 > rb) lb = i, rb = i + tz[i] - 1;
    }
    while (z[0] < s.size() and z[0] < t.size() and t[z[0]] == s[z[0]]) z[0]++;
    lb = 0, rb = z[0] - 1;
    for (int i = 1; i < s.size(); ++i) {
        if (i <= rb) z[i] = min(tz[i - lb], rb - i + 1);
        while (i + z[i] < s.size() and z[i] < t.size() and s[i + z[i]] == t[z[i]]) z[i]++;
        if (i + z[i] - 1 > rb) lb = i, rb = i + z[i] - 1;
    }
    return z;
}

```

4.3 Manacker algorithm

Given string s , $\forall i$ finds max n s.t. $s[:n] == s[i-n+1 : i+1]$. **TC:** $O(n)$ **SC:** $O(n)$

```

vi manacker_odd(string &s) {
    vi rad(s.size(), 1);
}

```

```

int lb = 0, rb = 0;
for (int i = 1; i < s.size(); ++i) {
    if (i <= rb) rad[i] = min(rad[lb + rb - i], rb - i + 1);
    while (0 <= i - rad[i] and i + rad[i] < s.size() and
           s[i - rad[i]] == s[i + rad[i]]) rad[i]++;
    if (rb < i + rad[i] - 1) lb = i - rad[i] + 1, rb = i + rad[i] - 1;
}
return rad;
}
vi manacker_even(string &s) {
    vi rad(s.size() - 1, 0);
    int lb = -1, rb = -1;
    for (int i = 0; i < s.size(); ++i) {
        if (i <= rb) rad[i] = min(rad[lb + rb - i - 1], rb - i);
        while (0 <= i - rad[i] and i + rad[i] - 1 < s.size() and
           s[i - rad[i]] == s[i + rad[i] - 1]) rad[i]++;
        if (rb < i + rad[i]) lb = i - rad[i] + 1, rb = i + rad[i];
    }
    return rad;
}

```

4.4 Hashing

$B >$ alphabet size, M – quiet large modulo. Try using 31, 53, 79, 89, 131 for B ; 970'592'641, 1e9+7 for M . Segment $[lb, rb]$ is inclusive.

```

int LIM;

struct Power {
    ll B, M;
    vector<ll> val;
    Power(ll B, ll M) : B(B), M(M), val(LIM + 1) {
        val[0] = 1;
        for (int i = 1; i <= LIM; ++i) val[i] = (val[i - 1] * B) % M;
    }
    ll operator()(size_t i) {
        return val[i];
    };
};

struct Hasher {
    Power &pw1, &pw2;
    vector<ll> ha1, ha2;
    Hasher(Power &pw1, Power &pw2, string &s)
    : pw1(pw1), pw2(pw2), ha1(s.size()), ha2(s.size()) {
        ha1[0] = (s[0] - 'a' + 1) * pw1(1);
        ha2[0] = (s[0] - 'a' + 1) * pw2(1);
        for (int i = 1; i < s.size(); ++i) {
            ha1[i] = (ha1[i - 1] + (s[i] - 'a' + 1) * pw1(i + 1)) % pw1.M;
            ha2[i] = (ha2[i - 1] + (s[i] - 'a' + 1) * pw2(i + 1)) % pw2.M;
        }
    }
    pll operator()(int lb, int rb) {
        auto halb = (lb == 0 ? 0 : ha1[lb - 1]), ha2lb = (lb == 0 ? 0 : ha2[lb - 1]);
        return {
            ((ha1[rb] + pw1.M - halb) % pw1.M) * pw1(LIM - rb) % pw1.M,
            ((ha2[rb] + pw2.M - ha2lb) % pw2.M) * pw2(LIM - rb) % pw2.M,
        };
    }
};

```

```

}
pll operator()(string &s) {
    ll h1 = (s[0] - 'a' + 1) * pw1(1);
    ll h2 = (s[0] - 'a' + 1) * pw2(1);
    for (int i = 1; i < s.size(); ++i) {
        h1 = (h1 + (s[i] - 'a' + 1) * pw1(i + 1)) % pw1.M;
        h2 = (h2 + (s[i] - 'a' + 1) * pw2(i + 1)) % pw2.M;
    }
    return {
        h1 * pw1(LIM - s.size() + 1) % pw1.M,
        h2 * pw2(LIM - s.size() + 1) % pw2.M,
    };
}
};

```

4.5 Prefix Tree

Efficiently stores strings as tree. **TC:** $O(\max(n_i))$ **SC:** $O(\sum n_i)$

```

int K = 26;

struct Node {
    char ch;
    int par;
    int idx = -1;
    int suff = -1;
    int exit = -1;
    vector<int> forward = vector(K, -1);
    vector<int> automat = vector(K, -1);
    Node(char ch = '^', int par = -1) : ch(ch), par(par) {}
};

```

```

struct Trie {
    vector<Node> buf = vector<Node>(1);
    void insert(string &s, int idx) {
        int b = 0;
        for (char ch : s) {
            int chi = ch - 'a';
            if (buf[b].forward[chi] == -1) {
                buf[b].forward[chi] = buf.size();
                buf.emplace_back(ch, b);
            }
            b = buf[b].forward[chi];
        }
        buf[b].idx = idx;
    }
    int search(string &s) {
        int b = 0;
        for (char ch : s) {
            int chi = ch - 'a';
            if (buf[b].forward[chi] == 0) return -1;
            b = buf[b].forward[chi];
        }
        return b;
    }
    int get_suff(int b) {
        if (buf[b].suff == -1) {
            if (b == 0 || buf[b].par == 0) buf[b].suff = 0;

```

```

        else
            buf[b].suff = get_transition(get_suff(buf[b].par), buf[b].ch);
        }
        return buf[b].suff;
    }
    int get_exit(int b) {
        if (buf[b].exit == -1) {
            if (b == 0 || buf[b].par == 0) buf[b].exit = 0;
            else if (buf[get_suff(b)].idx >= 0) buf[b].exit = get_suff(b);
            else buf[b].exit = get_exit(get_suff(b));
        }
        return buf[b].exit;
    }
    int get_transition(int b, char ch) {
        int chi = ch - 'a';
        if (buf[b].automat[chi] == -1) {
            if (buf[b].forward[chi] != -1) buf[b].automat[chi] = buf[b].forward[chi];
            else buf[b].automat[chi] = (b == 0 ? 0 : get_transition(get_suff(b), ch));
        }
        return buf[b].automat[chi];
    }
};

```

5 Graphs

5.1 Dijkstra Algorithm

Shortest path lengths to all vertices from src given all the edges are of positive cost

TC: $O(n \log m)$ **SC:** $O(\max(n, m))$

```

vi dijkstra(vector<vector<pii>> &graph, int src) {
    int n = graph.size();
    vi dist(n, INT32_MAX);
    dist[src] = 0;
    priority_queue<pii, vpri, greater<>> pq;
    pq.emplace(0, src);
    while (!pq.empty()) {
        while (!pq.empty() and dist[pq.top().S] <= pq.top().F) pq.pop();
        if (pq.empty()) continue;
        auto [d, v] = pq.top();
        pq.pop();
        dist[v] = d;
        for (auto [to, len] : graph[v]) pq.emplace(d + len, to);
    }
    return dist;
}

```

5.2 Shortest Path Faster

Shortest path lengths to all vertices from src with negative weights allowed. If negative cycle exists, false returned. **TC:** $O(m)$ for random graphs, $O(nm)$ in worst cases

```

pair<vi, bool> spf(vector<vector<pii>> &graph, int src) {
    int n = graph.size();
    vi dist(n, INT32_MAX), cnt(n, 0); dist[src] = 0;
    vector<bool> inqueue(n, false); inqueue[src] = true;
    queue<int> q; q.push(src);

```

```

while (!q.empty()) {
    int v = q.front(), d = dist[q.front()];
    q.pop();
    inqueue[v] = false;
    for (auto [to, len] : graph[v]) {
        if (d + len >= dist[to]) continue;
        dist[to] = d + len;
        if (inqueue[to]) continue;
        q.push(to);
        inqueue[to] = true;
        cnt[to]++;
        if (cnt[to] > n) return {dist, false}; // negative cycle
    }
}
return {dist, true};
}

```

5.3 Floyd-Warshall

Shortest paths between all pairs of vertices. The graph may have negative weights, but no negative weight cycles. **TC:** $O(n^3)$

```

vvi floyd_warshall(vvi &graph) {
    int n = graph.size();
    auto dist(graph);
    for (int i = 0; i < n; ++i) dist[i][i] = 0;
    for (int k = 0; k < n; ++k) {
        for (int i = 0; i < n; ++i) {
            for (int j = 0; j < n; ++j) {
                if (dist[i][k] != INT32_MAX and dist[k][j] != INT32_MAX and
                    dist[i][j] < dist[i][k] + dist[k][j]) {
                    dist[i][j] = dist[i][k] + dist[k][j];
                    dist[j][i] = dist[i][j];
                }
            }
        }
    }
    return dist;
}

```

5.4 Euler Path

Path containing all the edges. The graph must be connected and degrees of $\geq n - 2$ vertices must be even. Don't forget to reverse resulting vector. **TC:** $O(m)$

```

void euler(vector<hashset<int>> &graph, vi &path, int src = 0) {
    while (!graph[src].empty()) {
        int dst = *graph[src].begin();
        graph[src].erase(dst);
        graph[dst].erase(src); // only for directed graph
        euler(graph, path, dst);
    }
    path.push_back(src);
}

```

5.5 Hamilton Path

Path containing all the vertices. That's NP-complete problem so n must be really small.

TC: $O(2^n)$

```

int shortest_hamilton(int mask, int end, vvi &dp, vector<hashmap<int, int>> &graph) {
    if ((mask & (mask - 1)) == 0) return 0;
    if (dp[mask][end] != -1) return dp[mask][end];
    int len = INT32_MAX;
    for (int bit = 0; bit < 32; ++bit) {
        if ((mask & (1 << bit)) == 0) continue;
        if (graph[bit].count(end) == 0) continue;
        len = min(len, shortest_hamilton(mask & ~(1 << end), bit, dp, graph) +
            graph[bit].find(end)->second);
    }
    return dp[mask][end] = len;
}

int count_hamiltons(int mask, int end, vvi &dp, vector<hashset<int>> &graph) {
    if ((mask & (mask - 1)) == 0) return 1;
    if (dp[mask][end] != -1) return dp[mask][end];
    int cnt = 0;
    for (int bit = 0; bit < 32; ++bit) {
        if ((mask & (1 << bit)) == 0) continue;
        if (graph[bit].count(end) == 0) continue;
        cnt += count_hamiltons(mask & ~(1 << end), bit, dp, graph);
    }
    return dp[mask][end] = cnt;
}

int check_hamilton(int mask, vi &dp, vi &graph) {
    if ((mask & (mask - 1)) == 0) return mask;
    if (dp[mask] != -1) return dp[mask];
    int ans = 0;
    for (int bit = 0; bit < 32; ++bit) {
        if ((mask & (1 << bit)) == 0) continue;
        if ((check_hamilton(mask & ~(1 << bit), dp, graph) & graph[bit]) != 0)
            ans += 1 << bit;
    }
    return dp[mask] = ans;
}

```

5.6 Topological Sort

Orders vertices s.t. all edges go from left to right

```

vi toposort(vvi &graph) {
    vi order; order.reserve(graph.size());
    vector<bool> seen(graph.size(), false);
    for (int v = 0; v < graph.size(); ++v) if (!seen[v]) dfs(graph, seen, order, v);
    reverse(all(order));
    return order;
}

void dfs(vvi &graph, vector<bool> &seen, vi &order, int v) {
    seen[v] = true;
    for (int to: graph[v]) if (!seen[to]) dfs(graph, seen, order, to);
    order.push_back(v);
}

```


5.7 Strongly Connected Components

Splits graph into set of components, s.t. within one components there are paths in both directions between any two vertices. Compressing every component into single vertex makes graph acyclic. **TC:** $O(n)$

```
pair<vi, vi> scc(vvi &graph) {
    int n = graph.size();
    vvi trans(n);
    vi order; order.reserve(n);
    vector<bool> seen(n, false);
    for (int v = 0; v < n; ++v) {
        for (int to: graph[v]) trans[to].push_back(v);
        if (!seen[v]) dfs(graph, seen, order, v);
    }
    reverse(all(order));
    vi component(n, -1);
    int n_components = 0;
    for (int v: order)
        if (component[v] == -1) tdfs(trans, component, n_components, v), n_components++;
    return {component, order};
}

void dfs(vvi &graph, vector<bool> &seen, vi &order, int v) {
    seen[v] = true;
    for (int to: graph[v]) if (!seen[to]) dfs(graph, seen, order, to);
    order.push_back(v);
}

void tdfs(vvi &trans, vi &component, int n_components, int v) {
    component[v] = n_components;
    for (int to: trans[v])
        if (component[to] == -1) tdfs(trans, component, n_components, to);
}
```

5.8 Bridge Edges & Cut Vertices

Given undirected graph, finds edges s.t. removing each of them increases number of connected components. **TC:** $O(n + m)$

```
vvi graph; vector<bool> seen; vi tin, low; int timer = 0;
```

```
void find_bridges() {
    seen.assign(graph.size(), false);
    tin.assign(graph.size(), -1), low.assign(graph.size(), -1);
    for (int v = 0; v < graph.size(); ++v)
        if (!seen[v]) dfs(v);
}

void dfs(int v, int p = -1) {
    seen[v] = true;
    tin[v] = low[v] = timer++;
    for (int to: graph[v]) {
        if (to == p) continue;
        if (seen[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs(to, v);
            low[v] = min(low[v], low[to]);
            if (low[to] > tin[v]) ITS_BRIDGE(v, to);
        }
    }
}
```

Small change in dfs needed to find cut vertices s.t. removing each of them along with their edges increases number of connected components:

```
void dfs(int v, int p = -1) {
    seen[v] = true;
    tin[v] = low[v] = timer++;
    int children = 0;
    for (int to: graph[v]) {
        if (to == p) continue;
        if (seen[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs(to, v);
            low[v] = min(low[v], low[to]);
            if (low[to] > tin[v] and p != -1) ITS_CUTPOINT(v);
            children++;
        }
    }
    if (p == -1 and children > 1) ITS_CUTPOINT(v);
}
```

6 Math

6.1 Equations

$$ax^2 + bx + c = 0 \implies x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}, x_{extr} = -\frac{b}{2a}, y_{extr} = -\frac{b^2}{4a} + c$$

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix}^{-1} = \frac{1}{ad - bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$$

Given an equation $Ax = b$, find $x_i = \frac{\det A'_i}{\det A}$, where A'_i is A with i 'th column replaced by b

6.2 LU Decomposition

Matrix as product of lower triangular matrix and upper triangular matrix. **TC:** $O(n^3)$

```
pair<vii, vii> LU(vii &A) {
    int n = A.size();
    vii L(n, vi(n, 0)), U(n, vi(n, 0));
    for (int i = 0; i < n; ++i) L[i][i] = 1;
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            if (i <= j) {
                U[i][j] = A[i][j];
                for (int k = 0; k < i; ++k) U[i][j] -= L[i][k] * U[k][j];
            } else {
                L[i][j] = A[i][j];
                for (int k = 0; k < j; ++k) L[i][j] -= L[i][k] * U[k][j];
                L[i][j] /= U[j][j];
            }
        }
    }
    return {L, U};
}
```

6.3 Polynomials

Instantiate polynomial from its coefficients or from given point set using interpolation. Find it's root but beware that it may give more roots to the left or to the right

```
struct Poly {
    typedef double D;
    typedef vector<D> vd;
    vd cf;

    Poly(vd &_cf) : cf(_cf) {}
    Poly(vd &x, vd &y) {
        int n = x.size();
        cf.resize(n);
        vd temp(n);
        for (int k = 0; k < n - 1; ++k)
            for (int i = k + 1; i < n; ++i) y[i] = (y[i] - y[k]) / (x[i] - x[k]);
        double last = 0;
        temp[0] = 1;
        for (int k = 0; k < n; ++k) {
            for (int i = 0; i < n; ++i) {
                cf[i] += y[k] * temp[i];
                swap(last, temp[i]);
                temp[i] -= last * x[k];
            }
        }
    }
    double eval(double x) {
        double y = 0;
        for (int i = cf.size() - 1; i >= 0; --i) (y *= x) += cf[i];
        return y;
    }
    void diff() {
        for (int i = 1; i < cf.size(); ++i) cf[i - 1] = i * cf[i];
        cf.pop_back();
    }
    // may give more roots to left or to the right
    vd roots(double xmin, double xmax) {
        if (cf.size() == 2) return {-cf[0] / cf[1]};
        vd roots;
        Poly deriv(*this);
        deriv.diff();
        vd dr = deriv.roots(xmin, xmax);
        dr.push_back(xmin - 1);
        dr.push_back(xmax + 1);
        sort(dr.begin(), dr.end());
        for (int i = 0; i < dr.size() - 1; ++i) {
            double lb = dr[i], rb = dr[i + 1];
            bool sign = eval(lb) > 0;
            if (sign ^ (eval(rb) > 0)) {
                for (int it = 0; it < 60; ++it) {
                    double md = (lb + rb) / 2;
                    if ((eval(md) <= 0) ^ sign) lb = md;
                    else rb = md;
                }
                roots.push_back((lb + rb) / 2);
            }
        }
    }
}
```

```
    return roots;
}
};
```

6.4 Extremums

Finds minimum of f on $[a, b]$ where f has one local minima **TC:** $O((b - a)/\varepsilon)$

```
double gss(double a, double b, double (*)(double)) {
    double r = (sqrt(5) - 1) / 2, eps = 1e-6;
    double x1 = b - r * (b - a), x2 = a + r * (b - a);
    double f1 = f(x1), f2 = f(x2);
    while (b - a > eps) {
        if (f1 < f2) { // > for maximum
            b = x2; x2 = x1; f2 = f1;
            x1 = b - r * (b - a); f1 = f(x1);
        } else {
            a = x1; x1 = x2; f1 = f2;
            x2 = a + r * (b - a); f2 = f(x2);
        }
    }
    return a;
}
```

6.5 Sums

$$c^a + c^{a+1} + \dots + c^b = \frac{c^{b+1} - c^a}{c - 1}, \quad c \neq 1$$

$$1 + \dots + n = \frac{n(n+1)}{2} \quad 1^2 + \dots + n^2 = \frac{n(2n+1)(n+1)}{6}$$

$$1^3 + \dots + n^3 = \frac{n^2(n+1)^2}{4} \quad 1^4 + \dots + n^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$$

6.6 Trigonometry

$$\sin(a \pm b) = \sin a \cos b \pm \sin b \cos a \quad \cos(a \pm b) = \cos a \cos b \mp \sin a \sin b$$

$$\tan(a \pm b) = \frac{\tan a \pm \tan b}{1 \mp \tan a \tan b} \quad \cot(a \pm b) = \frac{\cot a \cot b \mp 1}{\cot a \pm \cot b}$$

$$\sin a \pm \sin b = 2 \sin \frac{a \pm b}{2} \cos \frac{a \mp b}{2} \quad \cos a + \cos b = 2 \cos \frac{a+b}{2} \cos \frac{a-b}{2}$$

$$(A+B) \frac{\tan(a-b)}{2} = (A-B) \frac{\tan(a+b)}{2}, \text{ where } A, B - \text{sides opposite to } a, b$$

$$(x, y) + \angle \varphi = (x \cos \varphi - y \sin \varphi, x \sin \varphi + y \cos \varphi)$$

6.7 Number theory

Fermat's little theorem

$$a \in \mathbb{Z}, p \in \mathbb{P} \implies a^p \equiv a \pmod{p}$$

Bézout's identity

$$a, b \in \mathbb{Z} \implies \exists x, y \in \mathbb{Z} : xa + yb = \gcd(a, b)$$

$$a, b \in \mathbb{N} \implies \exists x, y \in \mathbb{N} : xa - yb = \gcd(a, b)$$

if (x, y) – solution, then all the solutions: $\left(x + \frac{kb}{\gcd(a, b)}, y - \frac{ka}{\gcd(a, b)}\right), k \in \mathbb{Z}$

Euclid Extended

Finds gcd and Bézout coefficients

```
int euclid(int a, int b, int& x, int& y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    int x1, y1;
    int d = euclid(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
}
```

Chinese Remainder Theorem

Finds $x : x \equiv a \pmod{m}, x \equiv b \pmod{n}$. If $|a| < m, |b| < n \implies 0 \leq x < \text{lcm}(m, n)$. Assumes $mn < 2^{62}$. **TC:** $O(\log n)$

```
ll crt(ll a, ll m, ll b, ll n) {
    if (n > m) swap(a, b), swap(m, n);
    ll x, y; ll g = euclid(m, n, x, y);
    if ((a - b) % g != 0) return -1; // no solution
    x = (b - a) % n * x % n / g * m + a;
    return x < 0 ? x + m * n / g : x;
}
```

Modular Arithmetics

$$\star (a + b) \% m = ((a \% m) + (b \% m)) \% m$$

$$\star (a - b) \% m = ((a \% m) - (b \% m) + m) \% m$$

$$\star (a \cdot b) \% m = ((a \% m) \cdot (b \% m)) \% m$$

$$\star (b/a) \% m = ((b \% m) \cdot (a^{-1} \pmod{m})) \% m$$

$a^{-1} \pmod{m}$ is s.t $a \cdot (a^{-1} \pmod{m}) \equiv 1 \pmod{m}$. a^{-1} exists $\iff a, m$ are coprime ($\gcd(a, m) = 1$)

Long Modular Mul, Pow

For ull: $0 \leq a, b \leq m \leq 7.2 \cdot 10^{18}$

```
ull modmul(ull a, ull b, ull mod) {
    ll ret = a * b - mod * ull(1.L / mod * a * b);
    return ret + mod * (ret < 0) - mod * (ret >= (ll)mod);
}
```

```
ull modpow(ull a, ull b, ull mod) {
    ull ret = 1;
    for (; b; a = modmul(a, a, mod), b /= 2)
        if (b & 1) ret = modmul(ret, a, mod);
    return ret;
}
```

Inverting By Modulo

Computes $a^{-1} \pmod{m}$. **Note that m must be prime or coprime with a :**

```
ll modinv(ll a) {
    ll x, y;
    ll d = euclid(a, MOD, x, y);
    return (x + MOD) % MOD;
}
```

Primes

Miller-Rabin primality test for numbers $< 7 \cdot 10^{18}$. **TC:** $O(1)$

```
ull A[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022};
bool is_prime(ull num) {
    if (num < 2 or num % 6 % 4 != 1) return (num | 1) == 3;
    ull s = __builtin_ctzll(num - 1);
    ull d = num >> s;
    for (ull a: A) {
        ull p = modpow(a % num, d, num), i = s;
        while (p != 1 and p != num - 1 and a % num and i--)
            p = modmul(p, p, num);
        if (p != num - 1 and i != s) return false;
    }
    return true;
}
```

Generating primes

Eratosthenes sieve. Note that $S = (\text{int})\text{round}(\text{sqrt}(\text{LIM}))$

```
const int LIM = 1e6, S = 1e3;
bitset<LIM> is_prime;
vi eratosthenes() {
    const int R = LIM / 2;
    vi pr = {2}, sieve(S + 1);
    pr.reserve(int(LIM / log(LIM) * 1.1));
    vector<pii> cp;
    for (int i = 3; i <= S; i += 2) if (!sieve[i]) {
        cp.push_back({i, i * i / 2});
        for (int j = i * i; j <= S; j += 2 * i) sieve[j] = 1;
    }
    for (int L = 1; L <= R; L += S) {
        array<bool, S> block{};
        for (auto &[p, idx] : cp)
            for (int i = idx; i < S + L; idx = (i += p))
                block[i - L] = 1;
        for (int i = 0; i < min(S, R - L); ++i)
            if (!block[i]) pr.push_back((L + i) * 2 + 1);
    }
    for (int i : pr) is_prime[i] = 1;
    return pr;
}
```

Factorization

ρ -Pollard factorization, factors are returned in arbitrary order. **TC:** $O(\sqrt[4]{n})$

```
ull pollard(ull num) {
    auto f = [num](ull x) { return modmul(x, x, num) + 1; };
    ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
    while (t++ % 40 or __gcd(prd, num) == 1) {
        if (x == y) x = ++i, y = f(x);
        if ((q = modmul(prd, max(x, y) - min(x, y), num))) prd = q;
        x = f(x), y = f(f(y));
    }
    return __gcd(prd, num);
}
vector<ull> factor(ull num) {
    if (num == 1) return {};
    if (is_prime(num)) return {num};
    ull x = pollard(num);
    auto L = factor(x), R = factor(num / x);
    L.insert(L.end(), R.begin(), R.end());
    return L;
}
```

Divisibility

- * $n : 3$ (or 9) $\iff \sum\{\text{digits}\} : 3$ (or 9)
- * $n : 7 \iff (\sum\{\text{even digits}\} - \sum\{\text{odd digits}\}) : 7$
- * $n : 11 \iff (\sum\{\text{even digits}\} - \sum\{\text{odd digits}\}) : 11$ or $= 0$

Pythagorean triples

Given two numbers m, n : $m > n > 0$, m and n are coprime, $m - n$ is odd, generate triple:
 $a = m^2 - n^2, b = 2mn, c = m^2 + n^2$. In such way triples are unique. When multiplying
triples by $k > 0$, it's no longer guaranteed.

6.8 Combinatorics

Combinations

$$C_n^k = \frac{n!}{k!(n-k)!}$$

$$\overline{C_n^k} = C_{n+k-1}^k$$

$$C_n^0 = C_n^n = 1$$

$$C_n^k = C_n^{n-k}$$

$$C_{n-1}^{k-1} + C_{n-1}^k = C_n^k$$

$$C_n^k = \frac{n}{k} C_{n-1}^{k-1}$$

$$\sum_{k=0}^n C_n^k = 2^n$$

$$\sum_{r=0}^k C_m^r \cdot C_n^{k-r} = C_{n+m}^k$$

$$\sum_{k=0}^n C_r^k = C_{n+1}^{r+1}$$

$$\sum_{k=0}^n (C_n^k)^2 = C_{2n}^n$$

$$\sum_{k=0}^m (-1)^k \cdot C_n^k = (-1)^m \cdot C_{n-1}^m$$

Computes C_n^k modulo m . Precomputes factorials and reversed factorials. **TC:** $O(n)$

```
vector<ll> fac(LIM), rev(LIM);
fac[0] = 1;
for (int i = 1; i < LIM; i++) fac[i] = i * fac[i - 1] % MOD;
```

```
rev.back() = inv(fac.back());
for (int i = LIM - 1; i >= 1; i--) rev[i-1] = rev[i] * i % MOD;
ll C(int n, int k) { return fac[n] * rev[k] % MOD * rev[n - k] % MOD; }
```

Catalan numbers

$$C_0 = 1, C_n = \frac{1}{n+1} \binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n-1}, C_{n+1} = \frac{2(2n+1)}{n+2} C_n = \sum C_i C_{n-i}$$

- * Valid parentheses string of length $2n$
- * Binary trees with n internal nodes ($n+1$ leaves)
- * Paths from $(0, 0)$ to $(2n, 0)$ with steps $(1, 1)$, $(1, -1)$ above X-axis
- * Triangulations of polygon with $n+2$ vertices
- * Connections of $2n$ points on circle into n non-crossing chords
- * Decomposition of n into sum of positives (order matters)
- * Monopaths in grid $n \times n$ from left-top to right-bottom not crossing diagonal
- * Tuples $(x_1 \dots x_n) : x_1 = 1, x_i \leq x_{i-1} + 1$

7 Geometry

7.1 Points

Point in the plane. For better performance use `ll` instead of `double`, but remember about rounds and possible overflows. When comparing doubles remember about `EPS`.

```
typedef double D;
struct P {
    D x, y;
    P(D _x = 0, D _y = 0) : x(_x), y(_y) {}
    bool operator==(P p) { return tie(x, y) == tie(p.x, p.y); }
    P operator+(P p) const { return P(x + p.x, y + p.y); }
    P operator-(P p) const { return P(x - p.x, y - p.y); }
    P operator*(D d) const { return P(x * d, y * d); }
    P operator/(D d) const { return P(x / d, y / d); }
    D dot(P p) { return x * p.x + y * p.y; }
    D cross(P p) { return x * p.y - y * p.x; }
    D cross(P a, P b) { return (a - *this).cross(b - *this); }
    D dist2() { return x * x + y * y; }
    D dist() { return sqrt(dist2()); }
    double angle() { return atan2(y, x); }
    P unit() { return *this / dist(); }
    P perp() { return P(-y, x); }
    P normal() { return perp().unit(); }
    P rotate(double a) {
        return P(x * cos(a) - y * sin(a), x * sin(a) + y * cos(a));
    }
};
```

```
int sgn(D x) { return (x > 0) - (x < 0); }
static const D EPS = numeric_limits<D>::epsilon();
using VP = vector<P>;
```

Closest Points

The closest pair of points, i.e. with minimum distance.

```
pair<P, P> closest_points(VP ps) {
    assert(ps.size() > 1);
    set<P> S;
    sort(all(ps), [](P &a, P &b) { return a.y < b.y; });
    pair<D, pair<P, P>> clos{numeric_limits<D>::max(), {P(), P()}};
    int j = 0;
    for (auto &p : ps) {
        P d{1. + sqrt(clos.first), 0.};
        while (ps[j].y <= p.y - d.x) S.erase(ps[j++]);
        auto lo = S.lower_bound(p - d), hi = S.upper_bound(p + d);
        for (; lo != hi; ++lo) clos = min(clos, {(*lo - p).dist2(), {*lo, p}});
        S.insert(p);
    }
    return clos.second;
}
```

7.2 Lines

Point – Line distance

Signed distance from point p to line containing points s , e . Distance > 0 on the left side as seen from s towards e .

```
D line_dist(P s, P e, P p) {
    assert(s != e);
    return (e - s).cross(p - s) / (e - s).dist();
}
```

Point – Segment Distance

Shortest unsigned distance between point p and the line segment $[s, e]$.

```
D segment_dist(P s, P e, P p) {
    if (s == e) return (p - s).dist();
    D d = (e - s).dist2(), t = min(d, max(.0, (p - s).dot(e - s)));
    return ((p - s) * d - (e - s) * t).dist() / d;
}
```

Lines Intersection

If the lines $s1-e1$ and $s2-e2$ have single intersection point then $\{1, \text{point}\}$ returned. If no intersection point exists $\{0, (0, 0)\}$ returned. If infinitely many exist $\{-1, (0, 0)\}$ returned.

```
pair<int, P> line_isect(P s1, P e1, P s2, P e2) {
    auto d = (e1 - s1).cross(e2 - s2);
    if (d == 0) return {-(s1.cross(e1, s2) == 0), P(0, 0)};
    auto p = s2.cross(e1, s2), q = s2.cross(e2, s1);
    return {1, (s1 * p + e1 * q) / d};
}
```

Segments Intersection

If the line segments $[s1, e1]$ and $[s2, e2]$ have single intersection point then it's returned. If no intersection point exists an empty vector returned. If infinitely many exist a vector of 2 endpoints of common segment returned.

```
VP segment_isect(P s1, P e1, P s2, P e2) {
    D os1 = s2.cross(e2, s1), oe1 = s2.cross(e2, e1);
    D os2 = s1.cross(e1, s2), oe2 = s1.cross(e1, e2);
    if (sgn(os1) * sgn(oe1) < 0 and sgn(os2) * sgn(oe2) < 0)
        return {(s1 * oe1 - e1 * os1) / (oe1 - os1)};
    set<P> s;
    if (segment_dist(s2, e2, s1) <= EPS) s.insert(s1);
    if (segment_dist(s2, e2, e1) <= EPS) s.insert(e1);
    if (segment_dist(s1, e1, s2) <= EPS) s.insert(s2);
    if (segment_dist(s1, e1, e2) <= EPS) s.insert(e2);
    return {all(s)};
}
```

Side of Point

Where point p is as seen from s towards e : $1/0/-1 \iff \text{left/on line/right}$.

```
int side_of(P s, P e, P p) {
    D a = (e - s).cross(p - s);
    D q = (e - s).dist() * EPS;
    return (a > q) - (a < -q);
}
```

Linear Transformation

Apply the same linear transformation that takes line $s1-e1$ to $s2-e2$ to point p .

```
P transform(P s1, P e1, P s2, P e2, P p) {
    P d1 = e1 - s1, d2 = e2 - s2, num(d1.cross(d2), d1.dot(d2));
    return s2 + P((p - s1).cross(num), (p - s1).dot(num)) / d1.dist2();
}
```

7.3 Circles

Circles Intersection

The pair of points at which two circles intersect. Returns false in case of no intersection.

```
bool circle_isect(P c1, D r1, P c2, D r2, pair<P, P> *out) {
    if (c1 == c2) {
        assert(r1 != r2);
        return false;
    }
    P vec = c2 - c1;
    D d2 = vec.dist2(), sum = r1 + r2, dif = r1 - r2,
    p = (d2 + r1 * r1 - r2 * r2) / (d2 * 2), h2 = r1 * r1 - p * p * d2;
    if (sum * sum < d2 or dif * dif > d2) return false;
    P mid = c1 + vec * p, per = vec.perp() * sqrt(fmax(0, h2) / d2);
    *out = {mid + per, mid - per};
    return true;
}
```

Circles Tangents

The external tangents of two circles if $r2 \geq 0$ or internal if $r2 < 0$. Returns 0, 1 or 2 tangents.

```
vector<pair<P, P>> circle_tangents(P c1, D r1, P c2, D r2) {
    P d = c2 - c1;
    D dr = r1 - r2, d2 = d.dist2(), h2 = d2 - dr * dr;
    if (d2 == 0 or h2 < 0) return {};
    vector<pair<P, P>> tan;
```

```

for (D sign : {-1, 1}) {
    P v = (d * dr + d.perp() * sqrt(h2) * sign) / d2;
    tan.push_back({c1 + v * r1, c2 + v * r2});
}
if (h2 == 0) tan.pop_back();
return tan;
}

```

Intersection of Circle with Polygon

The area of the intersection of a circle with CCW polygon. **TC:** $O(n)$ **SC:** $O(1)$

```

#define arg(p, q) atan2((p).cross(q), (p).dot(q))
D circle_poly_isect(P c, D a, VP &pg) {
    auto tri = [&](P p, P q) {
        auto r2 = a * a / 2;
        P d = q - p;
        auto a = d.dot(p) / d.dist2(), b = (p.dist2() - a * a) / d.dist2();
        auto det = a * a - b;
        if (det <= 0) return arg(p, q) * r2;
        auto s = max(0., -a - sqrt(det)), t = min(1., -a + sqrt(det));
        if (t < 0 or 1 <= s) return arg(p, q) * r2;
        P u = p + d * s, v = p + d * t;
        return arg(p, u) * r2 + u.cross(v) / 2 + arg(v, q) * r2;
    };
    auto area = 0.;
    for (int i = 0; i < pg.size(); ++i)
        area += tri(pg[i] - c, pg[(i + 1) % pg.size()] - c);
    return area;
}

```

Circumcircle

The circle going intersecting all three points.

```

pair<P, D> circumcircle(P A, P B, P C) {
    P b = C - A, c = B - A, a = C - B;
    P cen = A + (b * c.dist2() - c * b.dist2()).perp() / b.cross(c) / 2;
    D rad = c.dist() * a.dist() * b.dist() / abs(c.cross(b)) / 2;
    return {cen, rad};
}

```

Minimum Enclosing Circle

The minimum circle that encloses a set of points. **TC:** $O(n)$ **SC:** $O(1)$

```

pair<P, D> enclosing_circle(VP &ps) {
    shuffle(all(ps), mt19937(time(0)));
    P cen = ps[0];
    D rad = 0, eps = 1 + 1e-8;
    for (int i = 0; i < ps.size(); ++i) {
        if ((cen - ps[i]).dist() > rad * eps) {
            cen = ps[i], rad = 0;
            for (int j = 0; j < i; ++j)
                if ((cen - ps[j]).dist() > rad * eps) {
                    cen = (ps[i] + ps[j]) / 2;
                    rad = (cen - ps[i]).dist();
                    for (int k = 0; k < j; ++k)
                        if ((cen - ps[k]).dist() > rad * eps) {
                            cen = circumcircle(ps[i], ps[j], ps[k]).first;

```

```

        rad = (cen - ps[i]).dist();
    }
}
return {cen, rad};
}

```

7.4 Polygons

Point Inside Polygon

Checks if p lies within the polygon. If `strict=false` then boundary isn't included. **TC:** $O(n)$

```

bool inside_poly(VP &pg, P a, bool strict = true) {
    int cnt = 0, n = pg.size();
    for (int i = 0; i < n; ++i) {
        P &q = pg[(i + 1) % n];
        if (segment_dist(pg[i], q, a) <= EPS) return !strict;
        cnt ^= ((a.y < pg[i].y) - (a.y < q.y)) * a.cross(pg[i], q) > 0;
    }
    return cnt;
}

```

Area of Polygon

Twice the signed area of CCW polygon. CW enumeration gives negative area.

```

D poly_area2(VP &pg) {
    D area = pg.back().cross(pg[0]);
    for (int i = 0; i < pg.size() - 1; ++i) area += pg[i].cross(pg[i + 1]);
    return area;
}

```

Centroid of Polygon

The center of mass for a polygon.

```

P poly_center(VP &pg) {
    P cen(0, 0);
    D area = 0;
    for (int i = 0, j = pg.size() - 1; i < pg.size(); j = i++) {
        cen = cen + (pg[i] + pg[j]) * pg[j].cross(pg[i]);
        area += pg[j].cross(pg[i]);
    }
    return cen / area / 3;
}

```

7.5 Hull

CCW convex hull, points on the edges aren't considered. **TC:** $O(n \log n)$ **SC:** $O(n)$

```

VP convex_hull(VP ps) {
    if (ps.size() <= 1) return ps;
    sort(all(ps));
    VP hull(ps.size() + 1);
    int s = 0, t = 0;
    for (int it = 2; it--; s = --t, reverse(all(ps)))
        for (auto p : ps) {

```

```

    while (t >= s + 2 and hull[t - 2].cross(hull[t - 1], p) <= 0) t--;
    hull[t++] = p;
}
return {hull.begin(), hull.begin() + t - (t == 2 and hull[0] == hull[1])};
}

```

Convex Hull Diameter

The two points with max distance on a CCW convex hull.

```

pair<P, P> hull_diam(VP &hull) {
    int n = hull.size(), j = n < 2 ? 0 : 1;
    pair<D, pair<P, P>> diam({0, {hull[0], hull[0]}});
    for (int i = 0; i < j; ++i) {
        for (; j = (j + 1) % n) {
            diam = max(diam, {(hull[i] - hull[j]).dist2(), {hull[i], hull[j]}});
            if ((hull[(j + 1) % n] - hull[j]).cross(hull[i + 1] - hull[i]) >= 0)
                break;
        }
    }
    return diam.second;
}

```

Point Inside Convex Hull

Checks if p lies within CCW convex hull. If strict=false then boundary isn't included.

TC: $O(\log n)$

```

bool inside_hull(VP &hull, P p, bool strict = true) {
    int a = 1, b = hull.size() - 1, r = !strict;
    if (hull.size() < 3)
        return r and segment_dist(hull[0], hull.back(), p) <= EPS;
    if (side_of(hull[0], hull[a], hull[b]) > 0) swap(a, b);
    if (side_of(hull[0], hull[a], p) >= r or side_of(hull[0], hull[b], p) <= -r)
        return false;
    while (abs(a - b) > 1) {
        int c = (a + b) / 2;
        (side_of(hull[0], hull[c], p) > 0 ? b : a) = c;
    }
    return sgn(hull[a].cross(hull[b], p)) < r;
}

```

7.6 Some Formulas

Triangles

Given side lengths a, b, c , vertices A, B, C , angles α, β, γ and semiperimeter $p = \frac{a+b+c}{2}$:

$$\text{Area } S = \sqrt{p(p-a)(p-b)(p-c)} \quad \text{Area } S = \frac{1}{2} |(A-C) \times (B-C)|$$

$$\text{Circumradius } R = \frac{abc}{4S} \quad \text{Inradius } r = \frac{S}{p}$$

$$\text{Median } m_a = \frac{1}{2} \sqrt{2b^2 + 2c^2 - a^2} \quad \text{Bisector } s_a = \sqrt{bc \left(1 - \frac{a^2}{(b+c)^2}\right)}$$

$$\text{Law of sines } \frac{\sin \alpha}{a} = \frac{\sin \beta}{b} = \frac{\sin \gamma}{c} = \frac{1}{2R} \quad \text{Law of cosines } a^2 = b^2 + c^2 - 2bc \cos \alpha$$

$$\text{Law of tangents } \frac{a+b}{a-b} = \frac{\tan \frac{\alpha+\beta}{2}}{\tan \frac{\alpha-\beta}{2}} \quad \text{Triple tangent } \frac{a}{\tan \alpha} + \frac{b}{\tan \beta} + \frac{c}{\tan \gamma} = 0$$

Quadrilaterals

Given side lengths a, b, c, d , diagonals e, f , diagonals angle θ , semiperimeter p and $F = b^2 + d^2 - a^2 - c^2$

$$4S = 2ef \sin \theta = F \tan \theta = \sqrt{4e^2 f^2 - F^2}$$

For cyclic quadrilaterals $ef = ac + bd$, $S = \sqrt{(p-a)(p-b)(p-c)(p-d)}$

Spherical coordinates

$$\begin{aligned} x &= r \sin \theta \cos \varphi & y &= r \sin \theta \sin \varphi & z &= r \cos \theta \\ r &= \sqrt{x^2 + y^2 + z^2} & \theta &= \arccos(z/r) & \varphi &= \operatorname{atan2}(y, x) \end{aligned}$$

7.7 Line Containers

Line Container

Container where you can add lines of the form $kx+m$, and query maximum values at point x. **TC:** $O(\log n)$

```

struct Line {
    mutable ll k, m, p;
    bool operator<(const Line& o) const { return k < o.k; }
    bool operator<(ll x) const { return p < x; }
};

struct LineContainer : multiset<Line, less<>> {
    // for doubles, use inf = 1/.0, div(a, b) = a/b
    static const ll inf = LLONG_MAX;
    ll div(ll a, ll b) { // floored division
        return a / b - ((a ^ b) < 0 && a % b); }
    bool isect(iterator x, iterator y) {
        if (y == end()) return x->p = inf, 0;
        if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
        else x->p = div(y->m - x->m, x->k - y->k);
        return x->p >= y->p;
    }
    void add(ll k, ll m) {
        auto z = insert({k, m, 0}), y = z++, x = y;
        while (isect(y, z)) z = erase(z);
        if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
        while ((y = x) != begin() && (--x)->p >= y->p) isect(x, erase(y));
    }
    ll query(ll x) {
        assert(!empty());
        auto l = *lower_bound(x);
        return l.k * x + l.m;
    }
};

```


Interval Container

Add and remove intervals from a set of disjoint intervals $[a, b]$. **TC:** $O(\log n)$

```
set<pii>::iterator addInterval(set<pii>& is, int L, int R) {
    if (L == R) return is.end();
    auto it = is.lower_bound({L, R}), before = it;
    while (it != is.end() && it->first <= R) {
        R = max(R, it->second);
        before = it = is.erase(it);
    }
    if (it != is.begin() && (--it)->second >= L) {
        L = min(L, it->first);
        R = max(R, it->second);
        is.erase(it);
    }
    return is.insert(before, {L, R});
}

void removeInterval(set<pii>& is, int L, int R) {
    if (L == R) return;
    auto it = addInterval(is, L, R);
    auto r2 = it->second;
    if (it->first == L) is.erase(it);
    else (int&)it->second = L;
    if (R != r2) is.emplace(R, r2);
}
```

Interval Cover

Compute indices of smallest set of intervals covering another interval $[a, b]$. If $[a, b]$, needed change (A) to add or `R.empty()`. Returns empty set on failure. **TC:** $O(n \log n)$

```
template<class T>
vi cover(pair<T, T> G, vector<pair<T, T>> I) {
    vi S(sz(I)), R;
    iota(all(S), 0);
    sort(all(S), [&](int a, int b) { return I[a] < I[b]; });
    T cur = G.first;
    int at = 0;
    while (cur < G.second) { // (A)
        pair<T, int> mx = make_pair(cur, -1);
        while (at < sz(I) && I[S[at]].first <= cur) {
            mx = max(mx, make_pair(I[S[at]].second, S[at]));
            at++;
        }
        if (mx.second == -1) return {};
        cur = mx.first;
        R.push_back(mx.second);
    }
    return R;
}
```

8 Dynamic Programming

8.1 Longest Increasing Subsequence

```
int find_lis(vi &nums) {
    if (nums.size() <= 1) return nums.size();
```

```
    vi lis(1, nums.front());
    for (int i = 1; i < nums.size(); ++i) {
        auto ins = lower_bound(lis.begin(), lis.end(), nums[i]);
        if (ins == lis.end()) lis.push_back(nums[i]);
        else *ins = nums[i];
    }
    return lis.size();
}
```

9 Various

9.1 Some numbers

- * INT32_MIN = -2'147'483'648
- * INT32_MAX = 2'147'483'647
- * UINT32_MAX = 4'294'967'295
- * INT64_MIN = -9'223'372'036'854'775'808
- * INT64_MAX = 9'223'372'036'854'775'807
- * UINT64_MAX = 18'446'744'073'709'551'615
- * (int)'a' = 97, (int)'z' = 122
- * (int)'A' = 65, (int)'Z' = 90
- * (int)'0' = 48, (int)'9' = 57
- * Primes: 31, 53, 79, 89, 131, 257, 1e9+7, 1e9+9, 2³¹-1, 1e18+3, 970'592'641, 31'443'539'979'727, 3'006'703'054'056'749

9.2 Bit Manipulations

Built-in binary operations for int's. With long long use suffix ll (except __lg)

- * __bit_floor – floor to 2's power
- * __bit_ceil – ceil to 2's power
- * __builtin_popcount – number of 1's
- * __builtin_parity – parity of number of 1's
- * __builtin_clz – number of leading zeros
- * __builtin_ctz – number of trailing zeros
- * __builtin_ffs – index of rightmost 1 (1-indexed)
- * __lg – floor(log2) = index of leftmost 1 (0-indexed)

Optimize popcount, clz via #pragma GCC target("popcnt,lzcnt")

Bit Hacks

- * `x & -x` – the least bit of `x`
- * `x & (x + 1)` – replace trailing 1's with 0's
- * `for (int x = m; x; x -= x &= m; ...)` – loop over all subset masks of `m`
- * `c = x & -x; r = x + c; ((r ^ x) >> 2) / c` | `r` – the next number after `x` with the same number of bits set

Vector Hashing

```
template <>
struct hash<vector<int>> {
    size_t operator()(const vector<int> &vec) const {
        size_t seed = vec.size();
        for(auto x : vec) {
            x = ((x >> 16) ^ x) * 0x45d9f3b;
            x = ((x >> 16) ^ x) * 0x45d9f3b;
            x = (x >> 16) ^ x;
            seed ^= x + 0x9e3779b9 + (seed << 6) + (seed >> 2);
        }
        return seed;
    }
};
```

Gray Code

Two successive numbers differ in only one bit

```
int to_graycode(int num) { return num ^ (num >> 1); }
```

9.3 Cycle Detection

Given the cyclic functional sequence finds cycle **TC:** $O(n)$ **SC:** $O(1)$

```
pii floyd_cycle(int x) {
    int tort = f(x), hare = f(f(x)), len = 1, pos = 0;
    while (tort != hare) tort = f(tort), hare = f(f(hare));
    tort = x;
    while (tort != hare) tort = f(tort), hare = f(hare), pos++;
    hare = f(hare);
    while (tort != hare) hare = f(hare), len++;
    return {len, pos}
}
```

```
pii brent_cycle(int x) {
    int tort = x, hare = f(x), pow = 1, len = 1, pos = 0;
    while (tort != hare) {
        if (pow == len) pow *= 2, len = 0, tort = hare;
        hare = f(hare), len++;
    }
    tort = hare = x;
    for (int i = 0; i < len; ++i) hare = f(hare);
    while (tort != hare) tort = f(tort), hare = f(hare), pos++;
    return {len, pos};
}
```